

SIMATS ENGINEERING

ASSESSMENT TOOL 2 (Debugging and Optimization Test)

COURSE CODE:CSA09

COURSE NAME: Programming in JAVA

COURSE OUTCOME COVERED

CO5. *Construct database-driven web applications by implementing JDBC connectivity to perform SQL operations such as insert, update, and delete using different statement types. (BL3, BL6)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:25

Annexure A

Debugging & Optimization Test

Code of Conduct:

*I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

Debugging & Optimization Test

You have to analyze the following buggy programs, identify issues, apply fixes, and optimize solutions by executing the code.

```
import java.sql.*;
public class UpdateEmployee {
    public static void main(String[] args) throws Exception {
        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/company", "root", "pass");

        String sql = "UPDATE employee SET salary=? WHERE id=?";
        PreparedStatement ps = con.prepareStatement(sql);

        ps.setDouble(1, 50000);
        ps.setInt(2, 101);

        int rows = ps.executeUpdate();
        System.out.println("Rows updated: " + rows);
    }
}
```

```
import java.sql.*;
public class DeleteStudent {
    public static void main(String[] args) throws Exception {
        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/school", "root", "pass");

        String sql = "DELETE FROM students WHERE id=?";
        PreparedStatement ps = con.prepareStatement(sql);

        ps.setInt(1, 1);

        int rows = ps.executeUpdate();
        System.out.println("Rows deleted: " + rows);
    }
}
```

```
import java.sql.*;
public class DisplayEmployees {
```

```

public static void main(String[] args) throws Exception {
    Connection con = DriverManager.getConnection(
        "jdbc:mysql://localhost:3306/company", "root", "pass");

    Statement st = con.createStatement();
    ResultSet rs = st.executeQuery("SELECT * FROM employee");

    while (rs.next()) {
        System.out.println(rs.getInt(1) + " " +
            rs.getString(2) + " " +
            rs.getDouble(3));
    }
}

import java.sql.*;
public class InsertProduct {
    public static void main(String[] args) throws Exception {
        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/store", "root", "pass");

        String sql = "INSERT INTO product VALUES(?, ?, ?)";
        PreparedStatement ps = con.prepareStatement(sql);

        ps.setInt(1, 101);
        ps.setString(2, "Laptop");
        ps.setDouble(3, 75000);

        ps.executeUpdate();
        System.out.println("Product inserted");
    }
}

import java.sql.*;
public class LoginCheck {
    public static void main(String[] args) throws Exception {
        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/appdb", "root", "pass");

        String username = "admin";
        String password = "1234";

        String sql = "SELECT * FROM users WHERE username=? AND password=?";

```

```
PreparedStatement ps = con.prepareStatement(sql);

ps.setString(1, username);
ps.setString(2, password);

ResultSet rs = ps.executeQuery();

if (rs.next()) {
    System.out.println("Login Successful");
} else {
    System.out.println("Invalid Credentials");
}
}
```

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation